Continues Integration

(gitlab ci)

О себе

Дзюба Дмитрий Владимирович

старший преподаватель к806

руководитель направления
«Архитектура» ООО Вымплком-ИТ

Примеры к занятию

План занятия

- **Введение**
- **Устройство Gitlab CI**
  - yaml
  - .gitlab-ci.yml
  - UI Gitlab
  - Gitlab Runner
- **Типовые примеры использования**

Введение

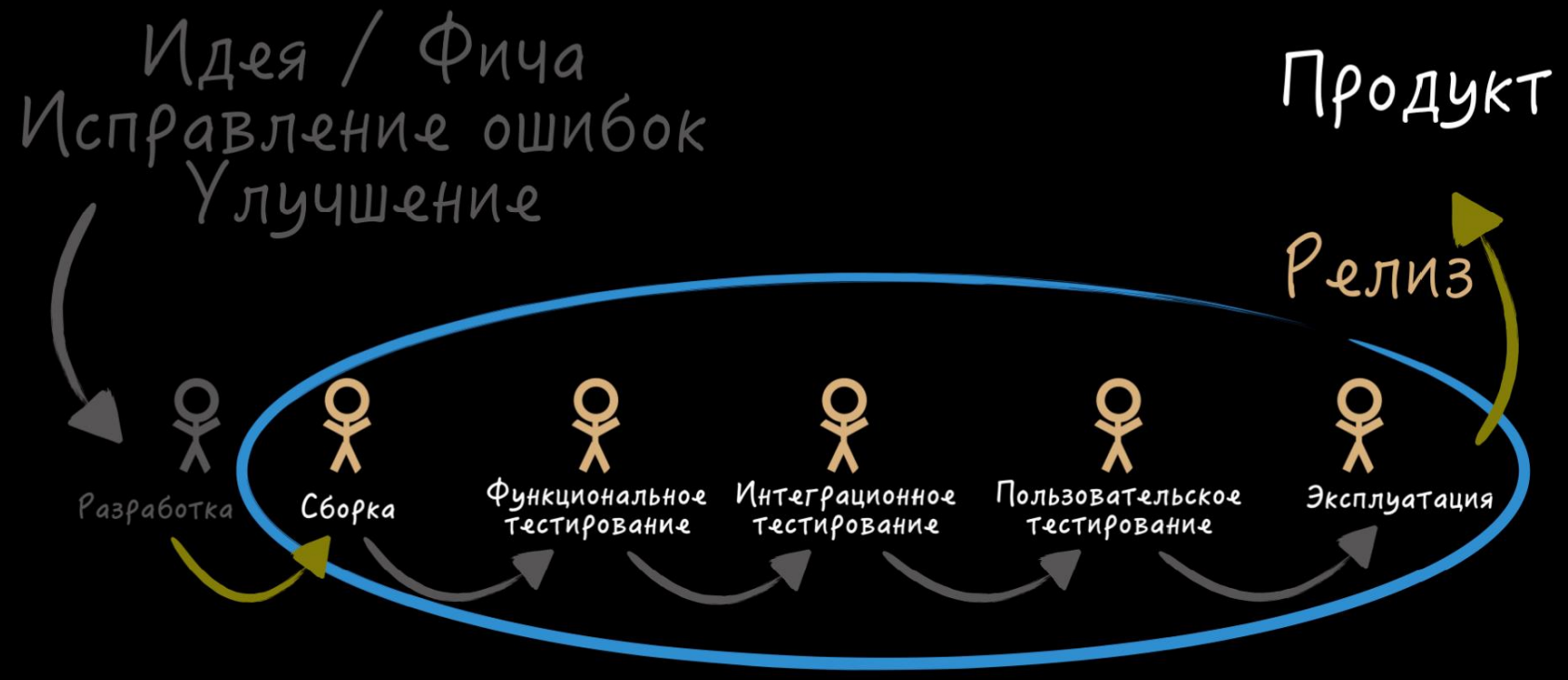
В опросе Forrester Research, посвященном управлению выпуском, на вопрос «Сколько времени уходит на внедрение изменения в одну строку кода?» более 80% опрошенных ответили, что данный процесс занимает более одного дня, при том что для 44% это время составляет неделю и больше.*

* - Five Ways To Streamline Release Management, Forrester Research Inc.

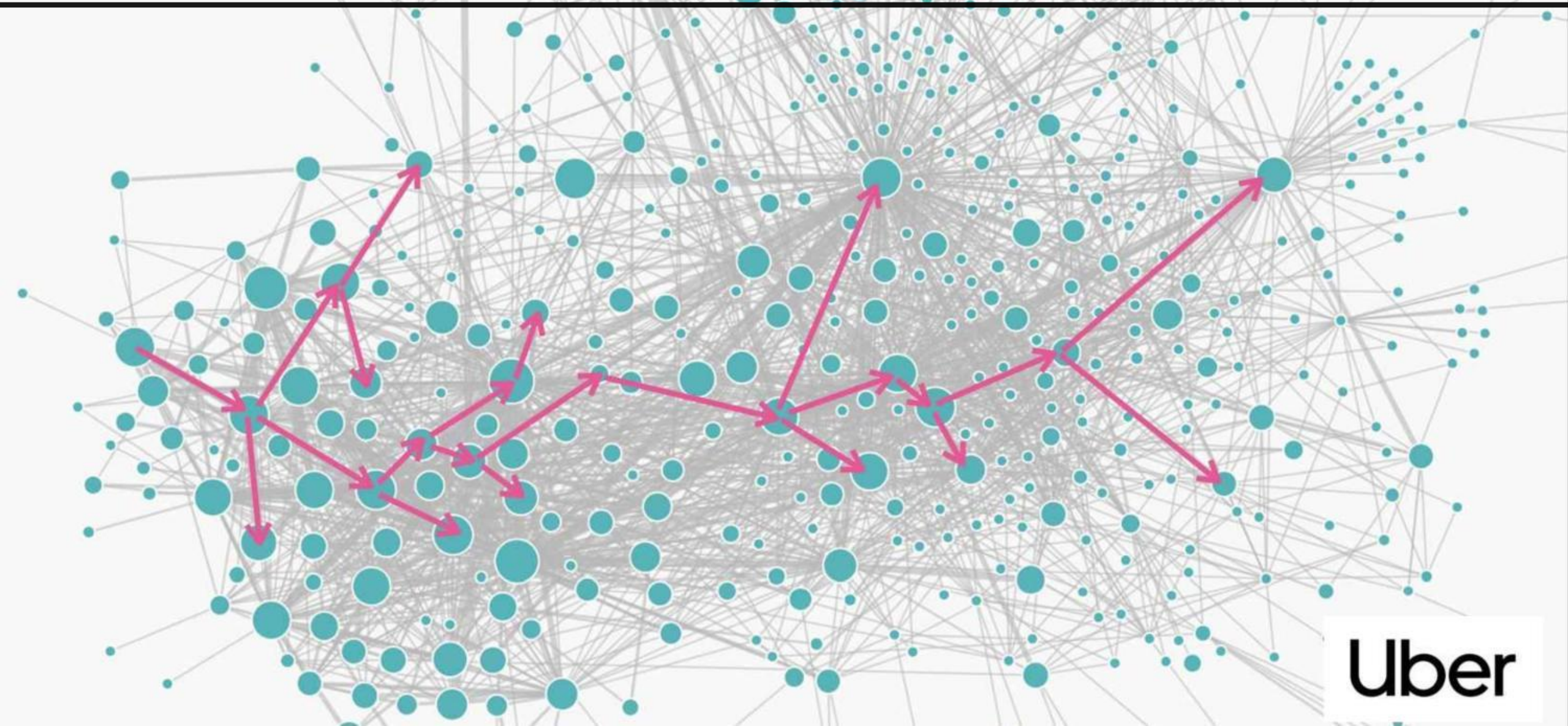
Разработка программного обеспечения

Цикл разработки

«Пусть перенесут на Production!»



Микросервисная архитектура



CI/CD

- **CI** (Continuous Integration) — это практика, при которой каждое изменение кода автоматически проходит тесты и сборку.
- **CD** (Continuous Delivery/Deployment) — это процесс автоматической доставки проверенного кода на сервер или в продакшн-среду.

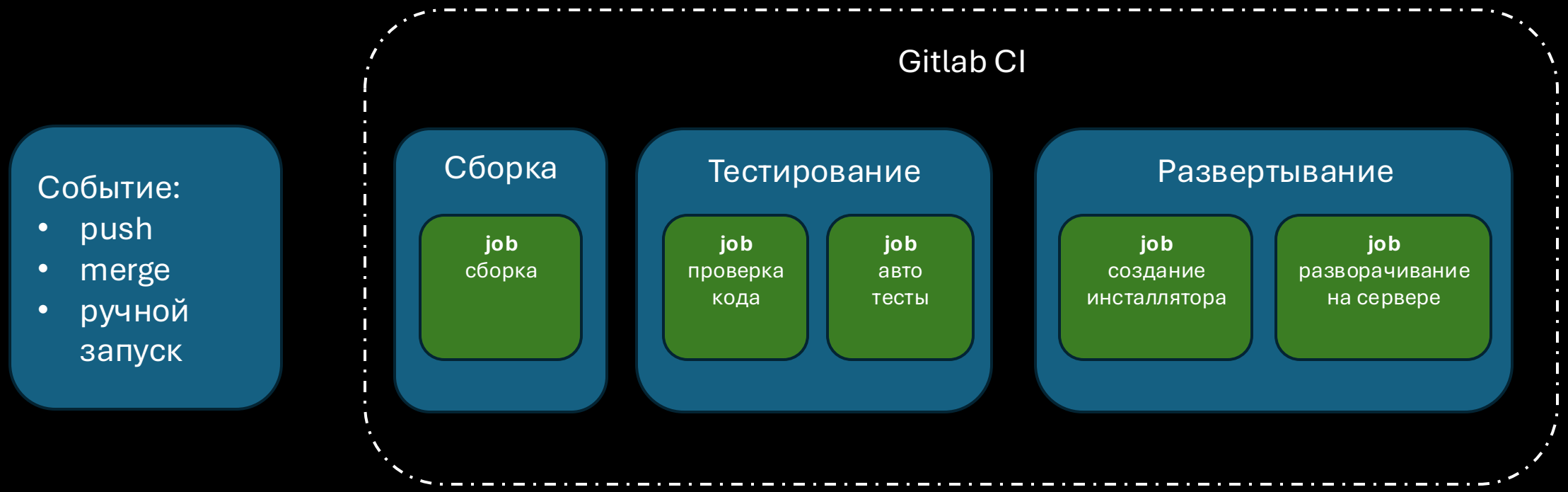
Цель CI/CD в GitLab

- ✓ автоматизировать рутину
- ✓ минимизировать ошибки
- ✓ ускорить релизы

Gitlab CI/CD

— полностью интегрированная в GitLab система для автоматизации сборки, тестирования и развертывания программного кода. GitLab CI/CD использует файл конфигурации **YAML** (язык разметки) в репозитории проекта для определения правил работы на каждом этапе в пайплайне. Поддерживает использование **Docker-**образов для определения окружения сборки — отсюда большая гибкость и повторное использование кода.

Пример CI/CD pipeline



.gitlab-ci.yml

- Хранит описание конфигурации Pipeline
- Находится в корне репозитория
- YAML файл

YAML

- понятный человеку код;
- минималистичный синтаксис;
- заточен под работу с данными;
- встроенный стиль, похожий на JSON (YAML является его надмножеством);
- поддерживает комментарии;
- поддерживает строки без кавычек;
- считается «чище», чем JSON;
- дополнительные возможности (расширяемые типы данных, относительные якоря и маппинг типов с сохранением порядка ключей).

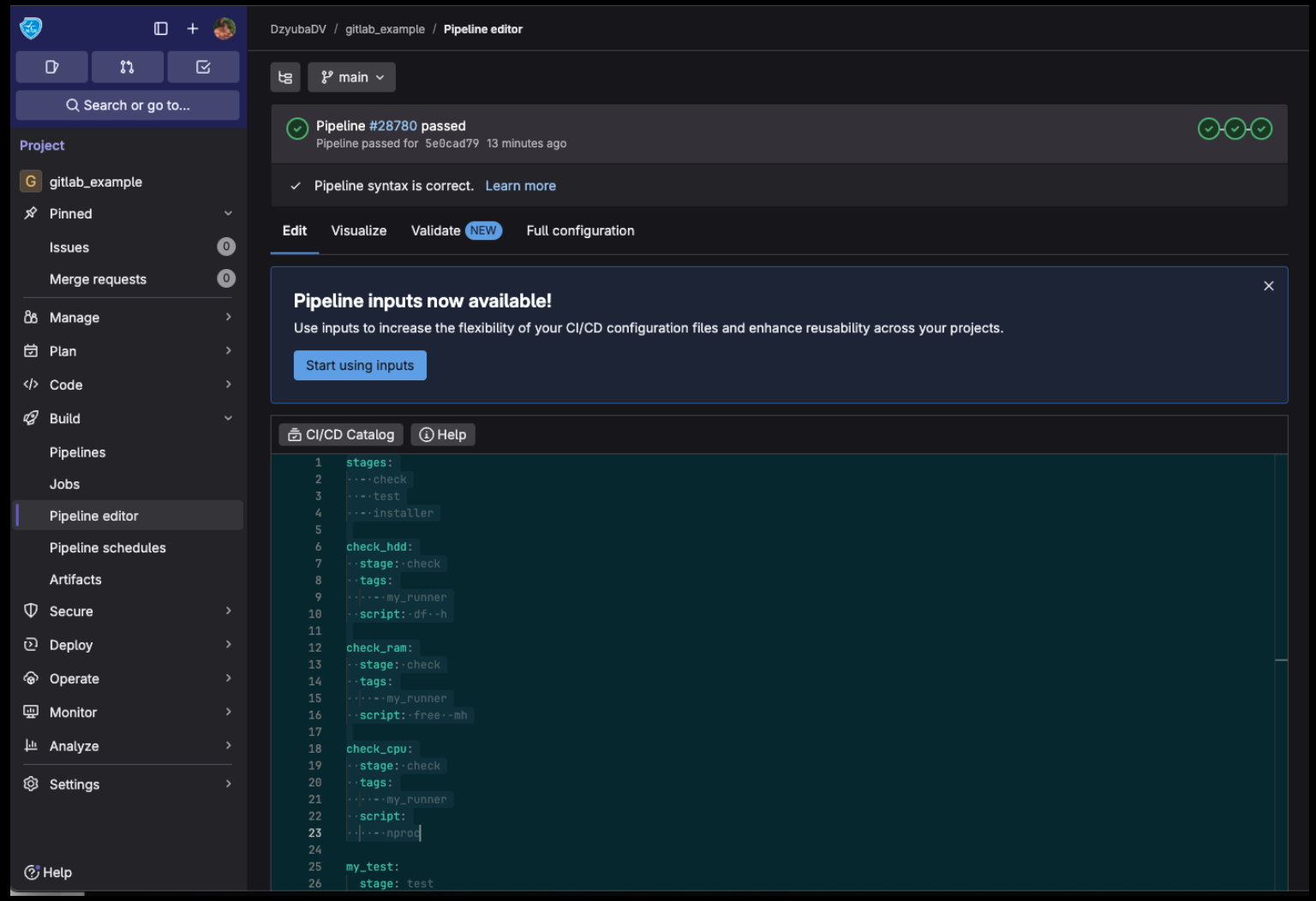
Пример

```yaml
#Comment: This is a supermarket list using YAML
#Note that - character represents the list
---
food:
  - vegetables: tomatoes #first list item
  - fruits: #second list item
      tropical: bananas
      nuts: peanuts
      sweets: raisins
```

Минимальный пример .gitlab-ci.yml

```yaml
stages:
  - build
  - test
  - deploy

build_job:
  stage: build
  script:
    - echo "Сборка проекта"

test_job:
  stage: test
  script:
    - echo "Запуск тестов"

deploy_job:
  stage: deploy
  script:
    - echo "Развертывание на сервере"
```

Gitlab CI UI

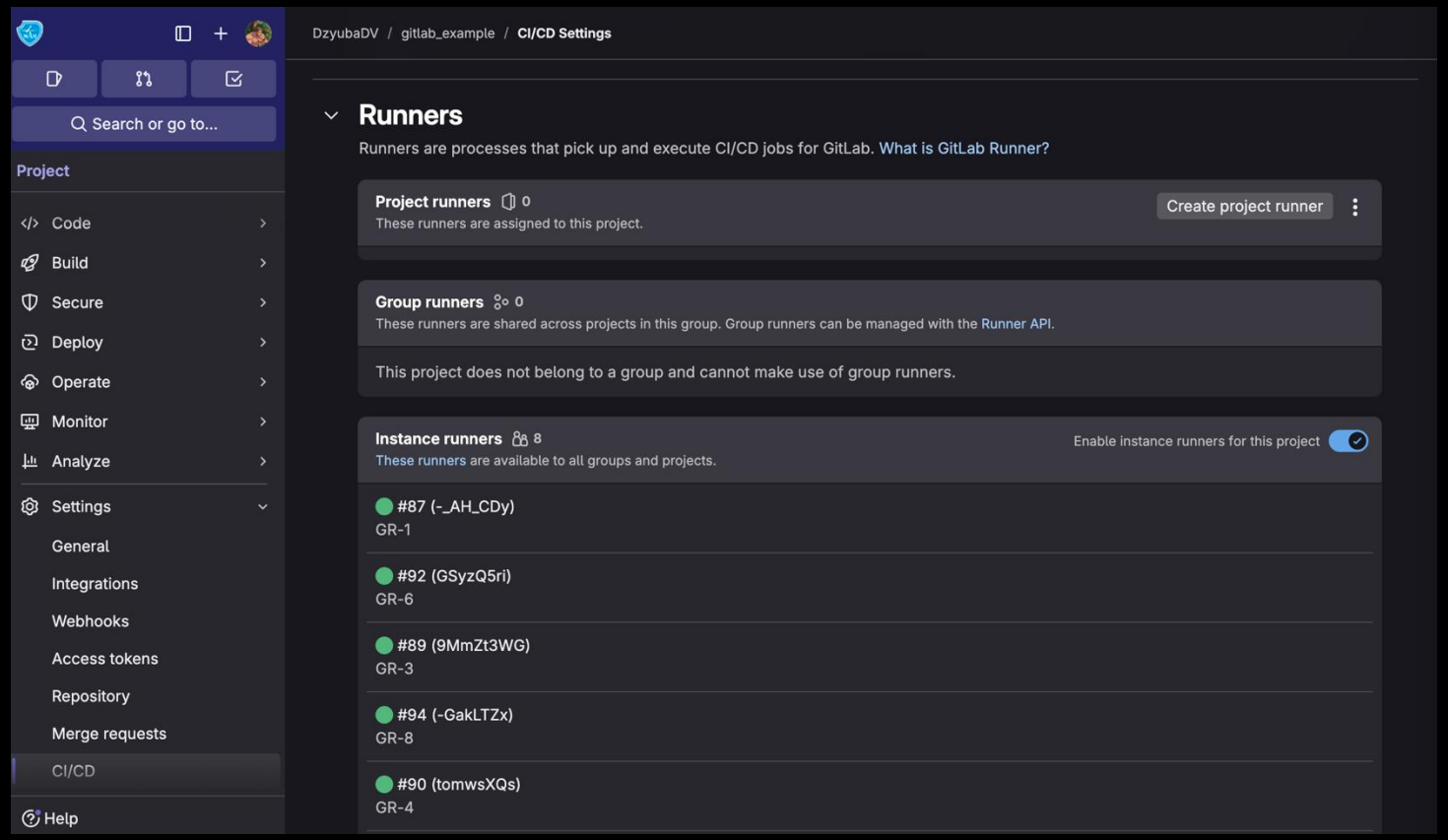

DzyubaDV / gitlab_example / Pipeline editor
main
Search or go to...
Project
gitlab_example
Pinned
Issues 0
Merge requests 0
Manage
Plan
Code
Build
Pipelines
Jobs
Pipeline editor
Pipeline schedules
Artifacts
Secure
Deploy
Operate
Monitor
Analyze
Settings
Help
Pipeline #28780 passed
Pipeline passed for 5e0cad79 13 minutes ago
Pipeline syntax is correct. Learn more
Edit
Visualize
Validate NEW
Full configuration
Pipeline inputs now available!
Use inputs to increase the flexibility of your CI/CD configuration files and enhance reusability across your projects.
Start using inputs
CI/CD Catalog
Help
stages:
  - check
  - test
  - installer
check_hdd:
  stage: check
  tags:
    - my_runner
  script: df -h
check_ram:
  stage: check
  tags:
    - my_runner
  script: free -mh
check_cpu:
  stage: check
  tags:
    - my_runner
  script:
    - nproc
my_test:
  stage: test

Определения

- **Пайплайн** — это весь процесс сборки, тестирования и развертывания, который запускается для каждого коммита.
- **Стадии** определяют последовательность выполнения (например, build, test, deploy). Задачи внутри одной стадии выполняются параллельно.
- **Задачи** — это отдельные шаги (например, build_job, test_job), в которых выполняются скрипты

Определение: **GitLab Runner**

— это отдельное приложение (агент), которое выполняет задачи пайплайна. Runner можно установить на собственном сервере или использовать бесплатные Shared Runner.



Пример .gitlab-ci.yml.00

```
job_cat:
  script:
    - result=$(cat file1.txt file2.txt | tr -d '\n')
    - |
      if [ "$result" != "Hello world" ]; then
        echo "Error: concatenation is '$result', expected 'Hello world'"
        exit 1
      else
        echo "Success: concatenation equals 'Hello world'"
      fi
```

Пример .gitlab-ci.yml.01

Стадия: Check

Стадия: test

```yaml
stages:
  - check
  - test

check_hdd:
  stage: check
  script: df -h

check_ram:
  stage: check
  script: free -mh

check_cpu:
  stage: check
  script:
    - nproc

my_test:
  stage: test
  script:
    - result=$(cat file1.txt file2.txt | tr -d '\n')
    - |
      if [ "$result" != "Hello world" ]; then
        echo "Error: concatenation is '$result', expected 'Hello world'"
        exit 1
      else
        echo "Success: concatenation equals 'Hello world'"
      fi
```

Артефакты и кэш

- **Артефакты** используются для передачи файлов (например, собранных бинарников) между задачами в разных стадиях одного пайплайна.
- **Кэш** используется для ускорения следующих пайплайнов, например, чтобы не загружать заново зависимости проекта

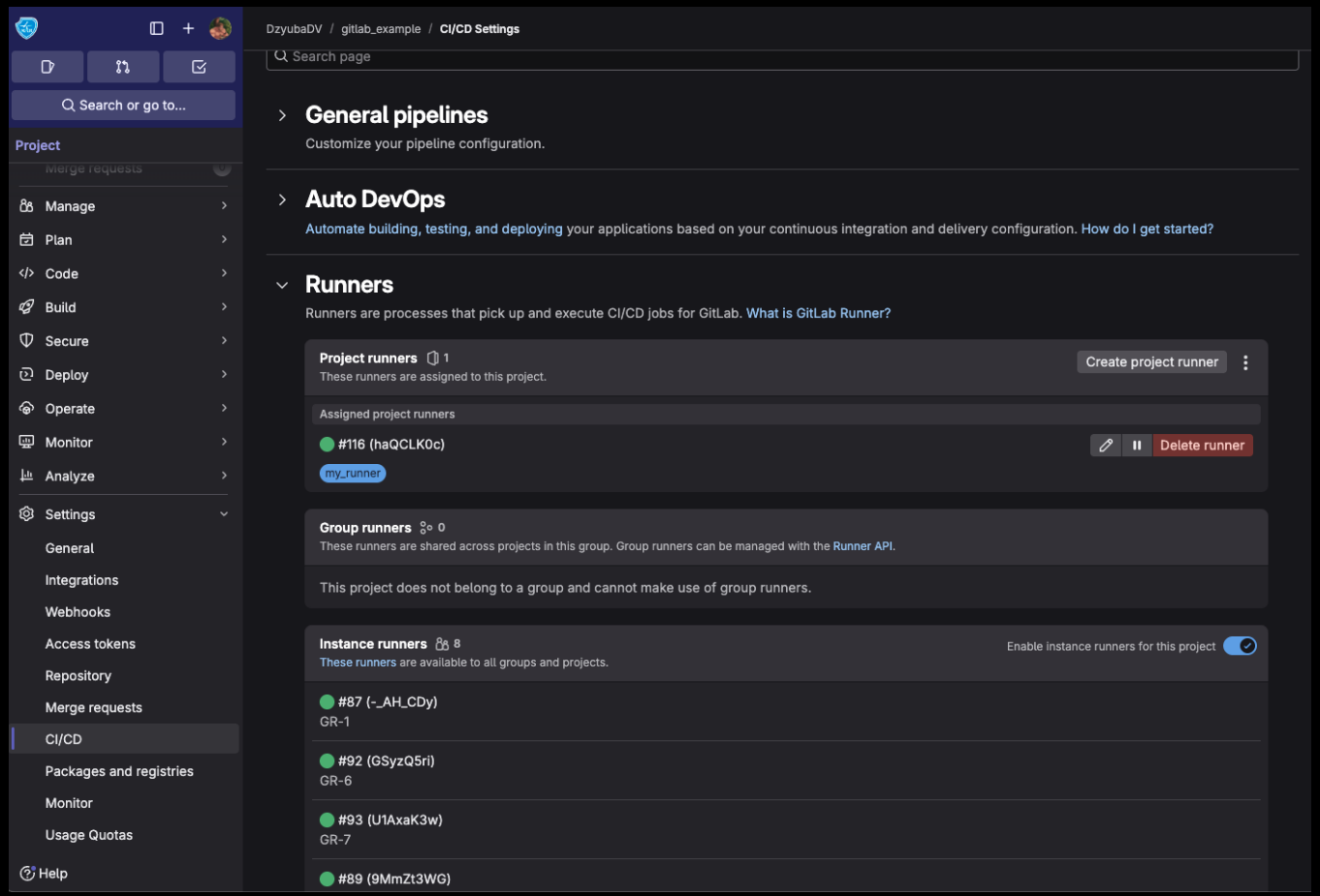
Пример **.gitlab-ci.yml.02**

```yaml
stages:
  - check
  - test
  - installer

...

package:
  image: ubuntu
  stage: installer
  script: tar -czf my_package.tar.gz ./*.txt
  artifacts:
    when: on_success
    access: all
    expire_in: 1 hour
    paths:
      - my_package.tar.gz
```

Стадия: installer

Запакуем файлы и поместим в хранилище артефактов pipeline

Регистрируем свой Runner



tag: my_runner

Регистрируем Runner на локальной машине

```
docker run --rm -it --name gitlab-runner -v ./config:/etc/gitlab-runner \
gitlab/gitlab-runner:latest register \
--url https://gitlab.mai.ru  --token CLK0cGuQbPXDrF8EHr286MQ
```

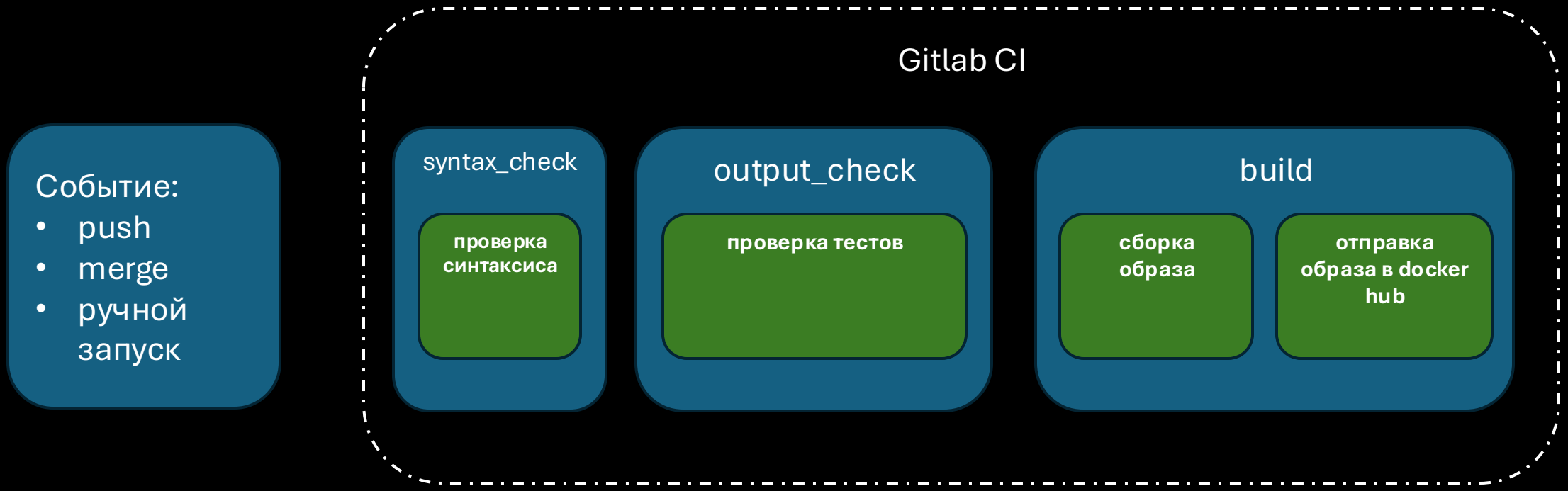
Запускаем runner на локальной машине

```
docker run -d --name gitlab-runner --restart always \
  -v ./config:/etc/gitlab-runner \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --privileged \
  gitlab/gitlab-runner:latest run
```

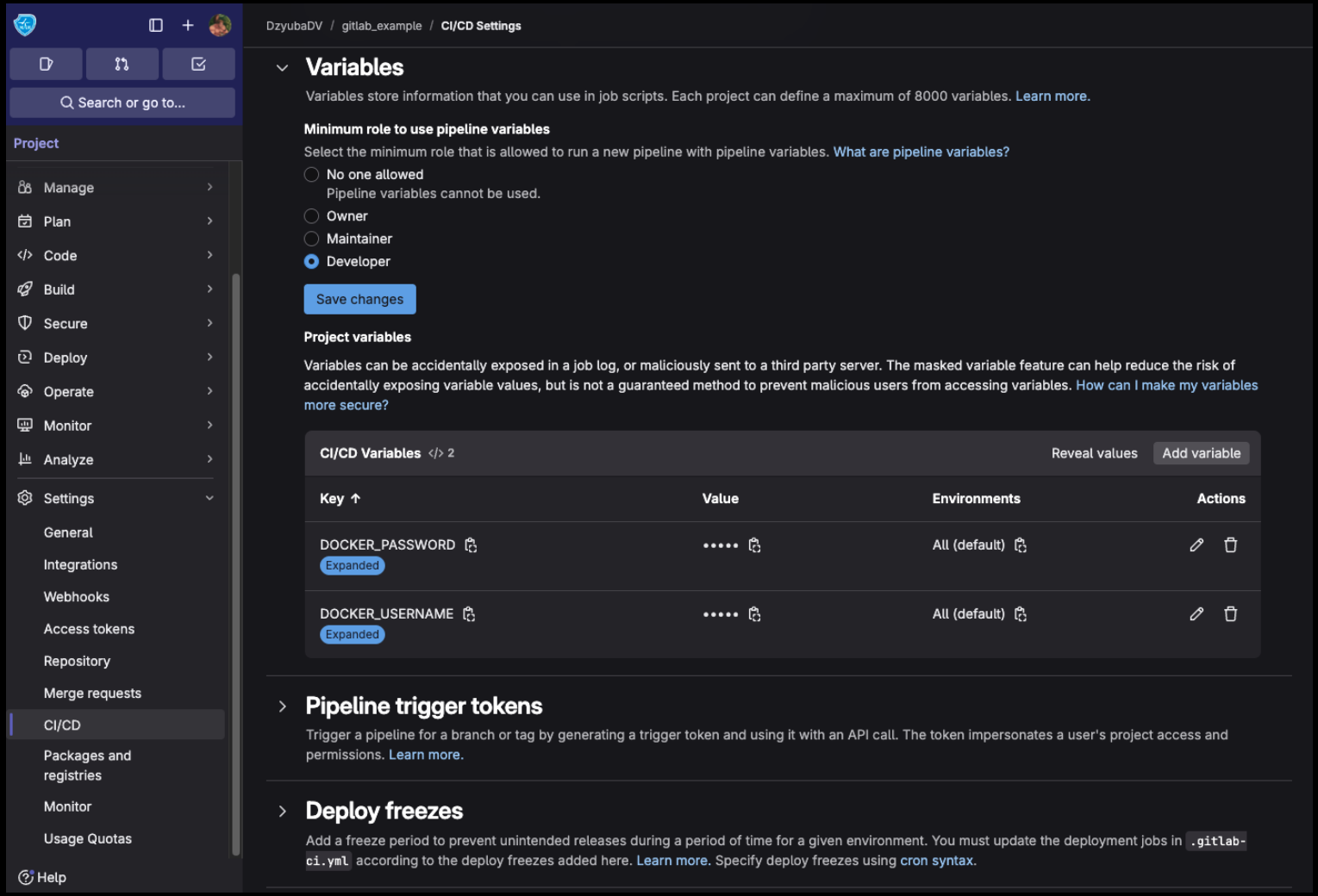
Добавляем tag my_runner .gitlab-ci.yml.03

```
stages:
  - check

check_hdd:
  stage: check
  tags:
    - my_runner
  script: df -h
```

Соберем пример приложения в Docker .gitlab-ci.yml.04



Настройка переменных



Итого Gitlab CI это

- "Автопилот" для твоего кода
- Личный телохранитель твоего кода
- Портфолио, которое ценится работодателями
- Скилл, который отделяет джуна от "почти мидла"